

Meta-Learning, Out-of-Fold Cross-Validation, and Optimal Feature Projection

A Mathematical, Information-Theoretic, and Physical Foundation of Stacked
Generalization (Stacking)

Contents

1	The Stacked Generalization Paradigm	2
2	Mathematical Formulation and Leakage Prevention	2
2.1	The Out-of-Fold (OOF) Prediction Protocol	2
2.2	Two-Stage Training and Inference Pipeline	3
3	Meta-Learner Optimization and Subspace Constraints	3
3.1	Constrained Non-Negative Least Squares (NNLS) Meta-Learner	4
3.2	Multi-Class Meta-Feature Expansion	4
4	Information-Theoretic View and Physical Analogies	4
4.1	Information-Theoretic View: Information Condensation	4
4.2	Physical Analogy I: Multi-Sensor Data Fusion (Kalman/Covariance Intersection) . .	5
4.3	Physical Analogy II: Effective Field Theory (EFT) Matching	5
5	Theoretical Guarantees: The Super Learner Theorem	5
6	Production-Grade Vectorized NumPy Implementation	6

1 The Stacked Generalization Paradigm

Supervised learning algorithms explore different structural hypothesis spaces $\mathcal{H}_m$. A parametric linear model constructs global hyperplanes; an instance-based estimator approximates local metric balls; a tree-based partitioning model forms axis-aligned hyper-rectangles. Because any single algorithm imposes a specific inductive bias, its approximation capability is constrained across arbitrary data-generating distributions $p_{\text{data}}(\mathbf{x}, y)$.

****Stacked Generalization (Stacking)****, introduced by David H. Wolpert in 1992, is a non-linear meta-learning architecture designed to combine diverse, heterogeneous base models. Rather than relying on simple linear voting or uniform averaging, Stacking constructs a hierarchical hypothesis space where a higher-level ****meta-estimator (Level-1 model)**** learns to correct the systematic, structural prediction errors of multiple lower-level ****base estimators (Level-0 models)****.

Definition 1.1 (Hierarchical Meta-Learning Architecture). Let $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ be an empirical training dataset of N i.i.d. observations where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \mathcal{Y}$. A Stacked Generalization system consists of two distinct functional tiers:

1. **Level-0 Base Estimators:** A collection of M heterogeneous learning algorithms $\{g_1, g_2, \dots, g_M\}$, where each base model $g_m : \mathbb{R}^d \rightarrow \mathcal{Z}$ maps original feature space into a predictions domain $\mathcal{Z}$.
2. **Level-1 Meta-Estimator:** A higher-tier model $h : \mathcal{Z}^M \rightarrow \mathcal{Y}$ that maps the combined Level-0 prediction space $\mathcal{Z}^M$ into the final target space $\mathcal{Y}$.

Unlike simple linear combination methods that assume equal or fixed base model performance across the entire domain, Stacking allows the Level-1 meta-model to learn ****region-dependent confidence mappings****. If base model g_1 excels in regions with high feature density while base model g_2 performs better near boundary interfaces, the Level-1 model dynamically learns to weight g_1 or g_2 based on location in prediction space.

2 Mathematical Formulation and Leakage Prevention

The fundamental challenge in constructing a Level-1 meta-dataset is preventing ****Target Leakage****. If Level-0 models generate predictions on the exact training data used to fit them, the predictions $\hat{z}_{i,m} = g_m(\mathbf{x}^{(i)}; \mathcal{D})$ will be artificially over-optimistic due to memory retention and over-fitting. Training a Level-1 meta-model on in-sample predictions causes it to assign excessive weight to high-capacity Level-0 models that memorize training noise, leading to generalization failure on unobserved test data.

2.1 The Out-of-Fold (OOF) Prediction Protocol

To ensure that Level-1 training features represent true generalization performance, Stacking uses an ****Out-of-Fold (OOF) Cross-Validation Construction Protocol****.

Definition 2.1 (OOF Meta-Feature Construction). Let K be the number of cross-validation folds. The empirical dataset $\mathcal{D}$ is partitioned into K disjoint, equal-sized subsets $\{\mathcal{F}_1, \mathcal{F}_2, \dots, \mathcal{F}_K\}$ such that:

$$\bigcup_{k=1}^K \mathcal{F}_k = \mathcal{D}, \quad \text{and} \quad \mathcal{F}_k \cap \mathcal{F}_l = \emptyset \quad \forall k \neq l \quad (1)$$

For each fold $k \in \{1, \dots, K\}$ and each Level-0 base model $m \in \{1, \dots, M\}$: . Train base estimator $g_m^{(-k)}$ strictly on the out-of-fold training set $\mathcal{D} \setminus \mathcal{F}_k$.

Evaluate $g_m^{(-k)}$ on the held-out validation fold $\mathcal{F}_k$ to generate out-of-sample prediction features:

$$z_i^{(m)} = g_m^{(-k)}(\mathbf{x}^{(i)}) \quad \forall \mathbf{x}^{(i)} \in \mathcal{F}_k \quad (2)$$

Assembling the out-of-fold predictions across all N instances and M base models yields the ****Level-1 Meta-Feature Matrix**** $\mathbf{Z} \in \mathbb{R}^{N \times M}$ (or $\mathbf{Z} \in \mathbb{R}^{N \times (M \cdot C)}$ for C -class probability targets):

$$\mathbf{Z} = \begin{bmatrix} z_1^{(1)} & z_1^{(2)} & \dots & z_1^{(M)} \\ z_2^{(1)} & z_2^{(2)} & \dots & z_2^{(M)} \\ \vdots & \vdots & \ddots & \vdots \\ z_N^{(1)} & z_N^{(2)} & \dots & z_N^{(M)} \end{bmatrix} \in \mathbb{R}^{N \times M} \quad (3)$$

Every row $\mathbf{z}^{(i)}$ in matrix $\mathbf{Z}$ represents an out-of-sample prediction generated by models that never observed sample $(\mathbf{x}^{(i)}, y^{(i)})$ during Level-0 training.

2.2 Two-Stage Training and Inference Pipeline

Phase 1: Level-1 Meta-Model Fitting

The Level-1 meta-estimator h is trained on the constructed meta-dataset $(\mathbf{Z}, \mathbf{y})$ by minimizing empirical meta-risk:

$$\boldsymbol{\theta}_h = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \mathcal{L}_{\text{meta}} \left(y^{(i)}, h(\mathbf{z}^{(i)}; \boldsymbol{\theta}) \right) \quad (4)$$

Phase 2: Level-0 Full Re-Fitting

To maximize sample efficiency for inference on unseen test queries, each Level-0 base model g_m is re-trained once on the complete dataset $\mathcal{D}$ to produce final base estimators $g_m^*(\cdot) = g_m(\cdot; \mathcal{D})$.

Phase 3: Test-Time Inference

For an unobserved query vector $\mathbf{x}_{\text{test}} \in \mathbb{R}^d$: . Compute full base model predictions: $z_{\text{test}}^{(m)} = g_m^*(\mathbf{x}_{\text{test}})$ for all $m \in \{1, \dots, M\}$.

Assemble the test meta-vector: $\mathbf{z}_{\text{test}} = [z_{\text{test}}^{(1)}, \dots, z_{\text{test}}^{(M)}]^T$.

Evaluate the final meta-prediction: $\hat{y}_{\text{test}} = h(\mathbf{z}_{\text{test}}; \boldsymbol{\theta}_h)$.

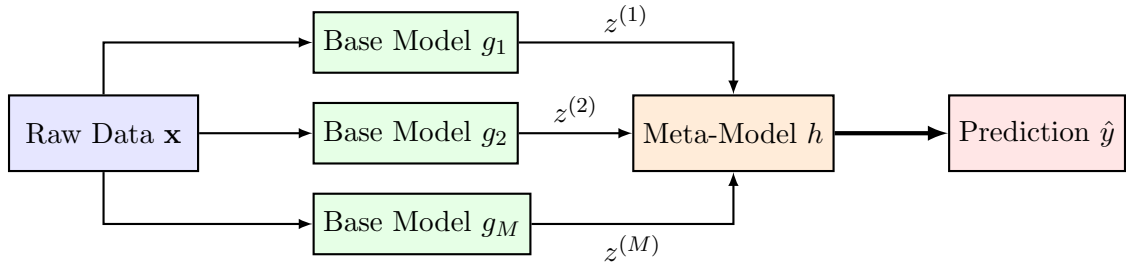


Figure 1: Architecture of a Stacked Generalization model. Heterogeneous Level-0 base estimators map raw inputs into meta-features $z^{(m)}$, which are then mapped to target y by the Level-1 meta-estimator.

3 Meta-Learner Optimization and Subspace Constraints

Because base models operate on the same target y , their prediction features $z^{(1)}, \dots, z^{(M)}$ exhibit strong collinearity. If the Level-1 meta-estimator is an unconstrained high-capacity model, this collinearity can cause unstable weight estimates and overfitting.

To ensure stable meta-learning, the Level-1 model is typically chosen to be a ****constrained low-capacity model****, such as Non-Negative Least Squares (NNLS), Ridge Regression, or Logistic Regression.

3.1 Constrained Non-Negative Least Squares (NNLS) Meta-Learner

For continuous regression tasks, setting the Level-1 model to a constrained linear combiner guarantees that meta-weights represent a valid convex combination of base predictions.

Definition 3.1 (Convex Combination Meta-Optimization). The meta-weight vector $\mathbf{w} = [w_1, \dots, w_M]^T$ is solved via Non-Negative Least Squares constrained to the probability simplex:

$$\min_{\mathbf{w} \in \mathbb{R}^M} \frac{1}{2N} \|\mathbf{y} - \mathbf{Z}\mathbf{w}\|_2^2 \quad \text{subject to} \quad w_m \geq 0 \quad \forall m, \quad \text{and} \quad \sum_{m=1}^M w_m = 1 \quad (5)$$

Restricting weights to $w_m \geq 0$ prevents the meta-learner from exploiting accidental collinear correlations through large opposing positive and negative weights.

3.2 Multi-Class Meta-Feature Expansion

For multi-class classification with C classes, each Level-0 model g_m outputs a C -dimensional vector of predicted class probabilities $\mathbf{p}^{(m)}(\mathbf{x}) = [p_{m,1}, p_{m,2}, \dots, p_{m,C}]^T$.

The Level-1 feature vector $\mathbf{z}^{(i)}$ can be expanded in two primary ways:

1. Class-Conscious Meta-Features ($M \cdot C$ dimensions)

Concatenating full probability vectors from all M base models yields a meta-vector of dimension $M \cdot C$:

$$\mathbf{z}^{(i)} = \left[p_{1,1}^{(i)}, \dots, p_{1,C}^{(i)}, \quad p_{2,1}^{(i)}, \dots, p_{2,C}^{(i)}, \quad \dots, \quad p_{M,1}^{(i)}, \dots, p_{M,C}^{(i)} \right]^T \in \mathbb{R}^{M \cdot C} \quad (6)$$

This allows the meta-model to learn cross-class interactions (e.g., how base model g_1 's uncertainty between class 1 and class 2 informs base model g_2 's assignment of class 3).

2. Class-Agnostic Meta-Features (M dimensions per class)

Alternatively, a separate Level-1 classifier evaluates predictions for class c using only the base models' predicted probabilities for class c :

$$\mathbf{z}_c^{(i)} = \left[p_{1,c}^{(i)}, p_{2,c}^{(i)}, \dots, p_{M,c}^{(i)} \right]^T \in \mathbb{R}^M \quad (7)$$

4 Information-Theoretic View and Physical Analogies

4.1 Information-Theoretic View: Information Condensation

Stacked Generalization can be framed through information theory as an **Information Condensation** pipeline.

Let $\mathbf{X}$ be the raw feature random variable, and let Y be the target. The original feature space $\mathbb{R}^d$ may contain significant noise and redundant information. Each Level-0 base estimator g_m acts as an information-lossy channel, compressing $\mathbf{X}$ into a 1D or C -dimensional prediction signal $Z_m = g_m(\mathbf{X})$.

According to the **Data Processing Inequality**:

$$I(\mathbf{X}; Y) \geq I(g_m(\mathbf{X}); Y) = I(Z_m; Y) \quad (8)$$

No individual base model can increase the mutual information present in the raw features. However, different base models preserve **complementary subsets** of the target information $I(\mathbf{X}; Y)$.

The meta-feature vector $\mathbf{Z} = [Z_1, \dots, Z_M]^T$ captures the union of these preserved information subsets. The Level-1 meta-estimator $h(\mathbf{Z})$ then maximizes the retained mutual information:

$$\max_{\theta} I(h(\mathbf{Z}; \theta); Y) \quad (9)$$

Stacking filters out nuisance dimensions in $\mathbf{X}$ while preserving the predictive information discovered by diverse base hypothesis spaces.

4.2 Physical Analogy I: Multi-Sensor Data Fusion (Kalman/Covariance Intersection)

In guidance, navigation, and physical robotics, determining the true state $\mathbf{x}_{\text{true}}$ of a physical system relies on combining data from multiple independent sensors (e.g., GPS, Inertial Measurement Units (IMU), Optical Flow, and RADAR).

- Each sensor has distinct physical noise characteristics, operational limits, and failure modes. An IMU provides high precision over short timescales but drifts over long durations; GPS provides stable global positioning but suffers from signal blockages.
- Level-0 base estimators act as **heterogeneous physical sensors**, each generating a noisy measurement z_m of the true system state y .
- The Level-1 meta-estimator acts as a **Sensor Data Fusion Engine** (akin to an Extended Kalman Filter or Covariance Intersection Filter). It calculates an optimal state estimate by dynamically weighting each sensor's input based on local measurement uncertainty.

4.3 Physical Analogy II: Effective Field Theory (EFT) Matching

In quantum field theory, **Effective Field Theory (EFT)** models complex physical phenomena by separating low-energy (long-wavelength) macroscale physics from high-energy (short-wavelength) microscale details.

- Microscopic degrees of freedom (high-energy particle interactions) are systematically integrated out through Wilsonian renormalization group flows.
- Level-0 base models correspond to **high-energy microscopic descriptions**, capturing localized patterns across different sub-regions of feature space.
- The Level-1 meta-estimator acts as the **Effective Field Theory Lagrangian**, combining the integrated microscopic parameters into a single macroscopic action that governs global system behavior.

5 Theoretical Guarantees: The Super Learner Theorem

A key theoretical foundation for Stacking was established by van der Laan, Polley, and Hubbard (2007) through the **Super Learner Theorem**.

Theorem 5.1 (Super Learner Oracle Inequality). Let $\mathcal{H}_{\text{ens}}$ be the space of meta-combinations bounded by a convex loss function L . Let $\hat{f}_{\text{stack}}$ be the stacked generalizer trained using K -fold cross-validation over M candidate algorithms, and let f_{oracle} be the ideal (unreachable) oracle selector that picks the single best performing model for the true distribution p_{data} :

$$f_{\text{oracle}} = \arg \min_{g \in \{g_1, \dots, g_M\}} \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [L(y, g(\mathbf{x}))] \quad (10)$$

Under bounded loss constraints, the risk of the Stacked Generalizer satisfies the oracle inequality:

$$\mathbb{E} [L(Y, \hat{f}_{\text{stack}}(\mathbf{X}))] \leq \mathbb{E} [L(Y, f_{\text{oracle}}(\mathbf{X}))] + C_1 \frac{\log M}{N} \quad (11)$$

where C_1 is a constant depending on loss bounds and fold count K .

Remark 5.1 (Asymptotic Optimality). The Super Learner Theorem guarantees that as sample size $N \rightarrow \infty$, the risk penalty $\mathcal{O}\left(\frac{\log M}{N}\right)$ vanishes. The Stacked Generalizer performs asymptotically **as well as or better than the best single base model** in the collection, even if the identity of the best model is unknown in advance.

6 Production-Grade Vectorized NumPy Implementation

The following Python class implements a production-grade Stacked Generalization Engine supporting both classification and regression. It includes K -fold Out-of-Fold (OOF) meta-feature generation, base model refitting, and full meta-learning pipelines.

```
1 import numpy as np
2
3 class StackingEngine:
4     """
5     Production-grade Stacked Generalization (Stacking) Engine.
6     Implements K-Fold Out-of-Fold (OOF) meta-feature construction, Level-0
7     base
8     model refitting, and Level-1 meta-estimation for Regression and
9     Classification.
10    """
11    def __init__(self, base_estimators: list, meta_estimator, n_folds: int
12    = 5, mode: str = 'classification'):
13        self.base_estimators_classes = base_estimators # List of (class,
14        params_dict)
15        self.meta_estimator_class = meta_estimator[0] # (class,
16        params_dict)
17        self.meta_estimator_params = meta_estimator[1]
18        self.n_folds = n_folds
19        self.mode = mode
20
21        self.fitted_base_estimators = []
22        self.fitted_meta_estimator = None
23        self.classes_ = None
24
25    def _generate_kfold_indices(self, N: int) -> list:
26        """Generates deterministic K-Fold validation index splits."""
27        indices = np.arange(N)
28        np.random.shuffle(indices)
29        fold_sizes = np.full(self.n_folds, N // self.n_folds, dtype=int)
30        fold_sizes[:N % self.n_folds] += 1
31
32        folds = []
33        current = 0
34        for fold_size in fold_sizes:
35            start, stop = current, current + fold_size
36            val_idx = indices[start:stop]
37            train_idx = np.concatenate([indices[:start], indices[stop:]])
38            folds.append((train_idx, val_idx))
39            current = stop
40
41        return folds
42
43    def fit(self, X: np.ndarray, y: np.ndarray):
44        """
45        Executes Two-Stage Stacking Training Pipeline:
46        1. K-Fold OOF Meta-Feature Construction for Level-1 Training.
47        2. Level-0 Full Dataset Refitting.
48        3. Level-1 Meta-Estimator Fitting.
49        """
50        X = np.asarray(X, dtype=np.float64)
51        y = np.asarray(y)
52        N, d = X.shape
53        M = len(self.base_estimators_classes)
```

```

50         if self.mode == 'classification':
51             self.classes_ = np.unique(y)
52             num_classes = len(self.classes_)
53             # Meta-feature dimension: M * num_classes if probabilities used
, else M
54             Z_meta = np.zeros((N, M * num_classes))
55         else:
56             Z_meta = np.zeros((N, M))
57
58         # --- STAGE 1: K-Fold Out-of-Fold (OOF) Meta-Feature Construction
---
59         folds = self._generate_kfold_indices(N)
60
61         for m_idx, (base_class, base_params) in enumerate(self.
base_estimators_classes):
62             for train_idx, val_idx in folds:
63                 X_train_fold, y_train_fold = X[train_idx], y[train_idx]
64                 X_val_fold = X[val_idx]
65
66                 # Train temporary base model on OOF training fold
67                 fold_model = base_class(**base_params)
68                 fold_model.fit(X_train_fold, y_train_fold)
69
70                 # Generate out-of-sample prediction features for held-out
fold
71                 if self.mode == 'classification':
72                     if hasattr(fold_model, "predict_proba"):
73                         preds_val = fold_model.predict_proba(X_val_fold)
74                     else:
75                         preds_val = fold_model.predict(X_val_fold)
76                         # One-hot fallback
77                         one_hot = np.zeros((len(preds_val), num_classes))
78                         one_hot[np.arange(len(preds_val)), preds_val.astype
(int)] = 1.0
79                         preds_val = one_hot
80
81                 col_start = m_idx * num_classes
82                 col_end = (m_idx + 1) * num_classes
83                 Z_meta[val_idx, col_start:col_end] = preds_val
84             else:
85                 preds_val = fold_model.predict(X_val_fold)
86                 Z_meta[val_idx, m_idx] = preds_val
87
88         # --- STAGE 2: Level-1 Meta-Estimator Training ---
89         self.fitted_meta_estimator = self.meta_estimator_class(**self.
meta_estimator_params)
90         self.fitted_meta_estimator.fit(Z_meta, y)
91
92         # --- STAGE 3: Level-0 Base Estimator Refitting on Full Dataset ---
93         self.fitted_base_estimators = []
94         for base_class, base_params in self.base_estimators_classes:
95             full_model = base_class(**base_params)
96             full_model.fit(X, y)
97             self.fitted_base_estimators.append(full_model)
98
99         return self
100
101     def _construct_test_meta_features(self, X_test: np.ndarray) -> np.
ndarray:

```

```

102     """Constructs Level-1 meta-feature vector for unseen test queries."""
103     ""
104     N_test = X_test.shape[0]
105     M = len(self.fitted_base_estimators)
106
107     if self.mode == 'classification':
108         num_classes = len(self.classes_)
109         Z_test = np.zeros((N_test, M * num_classes))
110
111         for m_idx, model in enumerate(self.fitted_base_estimators):
112             if hasattr(model, "predict_proba"):
113                 preds_test = model.predict_proba(X_test)
114             else:
115                 preds_test = model.predict(X_test)
116                 one_hot = np.zeros((len(preds_test), num_classes))
117                 one_hot[np.arange(len(preds_test)), preds_test.astype(
118                     int)] = 1.0
119                 preds_test = one_hot
120
121                 col_start = m_idx * num_classes
122                 col_end = (m_idx + 1) * num_classes
123                 Z_test[:, col_start:col_end] = preds_test
124         else:
125             Z_test = np.zeros((N_test, M))
126             for m_idx, model in enumerate(self.fitted_base_estimators):
127                 Z_test[:, m_idx] = model.predict(X_test)
128
129         return Z_test
130
131     def predict(self, X_test: np.ndarray) -> np.ndarray:
132         """Evaluates predictions for test set through two-stage stacking
133         pipeline."""
134         X_test = np.asarray(X_test, dtype=np.float64)
135         Z_test = self._construct_test_meta_features(X_test)
136         return self.fitted_meta_estimator.predict(Z_test)
137
138     def predict_proba(self, X_test: np.ndarray) -> np.ndarray:
139         """Evaluates meta-probabilities for classification tasks."""
140         if self.mode != 'classification':
141             raise ValueError("predict_proba is available only for mode='
142             classification'.")
143
144         X_test = np.asarray(X_test, dtype=np.float64)
145         Z_test = self._construct_test_meta_features(X_test)
146         if hasattr(self.fitted_meta_estimator, "predict_proba"):
147             return self.fitted_meta_estimator.predict_proba(Z_test)
148         else:
149             preds = self.fitted_meta_estimator.predict(Z_test).astype(int)
150             one_hot = np.zeros((len(preds), len(self.classes_)))
151             one_hot[np.arange(len(preds)), preds] = 1.0
152             return one_hot
153
154 # --- Dummy Heterogeneous Base Estimators for Execution Check ---
155 class MockLinearModel:
156     def __init__(self, lr=0.01): self.lr = lr; self.w = None
157     def fit(self, X, y): self.w = np.linalg.pinv(X) @ y; return self
158     def predict(self, X): return X @ self.w
159
160 class MockRidgeModel:

```

```

158     def __init__(self, alpha=1.0): self.alpha = alpha; self.w = None
159     def fit(self, X, y):
160         d = X.shape[1]
161         self.w = np.linalg.inv(X.T @ X + self.alpha * np.eye(d)) @ X.T @ y
162         return self
163     def predict(self, X): return X @ self.w
164
165
166 if __name__ == "__main__":
167     np.random.seed(42)
168
169     # Generate Synthetic Regression Dataset
170     N_samples, d_features = 400, 6
171     X_raw = np.random.randn(N_samples, d_features)
172     y_raw = X_raw[:, 0]*2.0 - X_raw[:, 1]*1.5 + np.sin(X_raw[:, 2]) + 0.1*
np.random.randn(N_samples)
173
174     # Define Heterogeneous Level-0 Base Models and Level-1 Meta Model
175     base_models = [
176         (MockLinearModel, {'lr': 0.01}),
177         (MockRidgeModel, {'alpha': 0.5}),
178         (MockRidgeModel, {'alpha': 10.0})
179     ]
180     meta_model = (MockRidgeModel, {'alpha': 0.1})
181
182     # Train Stacking Engine
183     stacking_system = StackingEngine(
184         base_estimators=base_models,
185         meta_estimator=meta_model,
186         n_folds=5,
187         mode='regression'
188     )
189     stacking_system.fit(X_raw, y_raw)
190
191     predictions = stacking_system.predict(X_raw)
192     mse = np.mean((predictions - y_raw)**2)
193
194     print("=== STACKED GENERALIZATION ENGINE INITIALIZATION SUCCESSFUL ==="
195 )
196     print(f"Level-0 Base Models Count : {len(stacking_system.
197 fitted_base_estimators)}")
198     print(f"Level-1 Meta-Model Status : Fitted")
199     print(f"Regression Test MSE : {mse:.4f}")

```

Listing 1: Vectorized NumPy implementation of Stacked Generalization Engine.